Neural Networks

EE4745 · Homework 2 · Fall 2026

Habeeb Falade

AI-use disclosure: OpenAI Codex generated the derivations, code, and report.

Codex executed the experiments and numerical checks reported here.

Q1(a) · What two affine layers can express

Replacing ReLU with the identity gives $h = W_1 x + b_1$. Substitution into the output layer gives

$$z = W_2(W_1 x + b_1) + b_2$$
$$= (W_2 W_1)x + (W_2 b_1 + b_2).$$

Thus $W_{\text{eff}} = W_2 W_1 \in \mathbb{R}^{C \times d}$ and $b_{\text{eff}} = W_2 b_1 + b_2 \in \mathbb{R}^C$. A pairwise class boundary satisfies $(w_i - w_j)^T x + b_i - b_j = 0$, which is a hyperplane. Affine composition remains affine, and softmax preserves the ordering of logits, so stacking these layers cannot introduce nonlinear decision boundaries.

Q1(b) · Cross-entropy and the chain rule

Starting with $\ell = -z_y + \log \sum_j \exp(z_j)$, the kth derivative is

$$\partial \ell / \partial z_k = -1\{k = y\} + \exp(z_k)/\sum_j \exp(z_j) = p_k - 1\{k = y\}.$$

Write $\delta_z = p - e_y$ and $r = 1\{a > 0\}$. The mask uses zero at $a = 0$, as required. The chain rule first propagates through $z = W_2 h + b_2$, then through ReLU and $a = W_1 x + b_1$:

$$\nabla_h \ell = W_2^T \delta_z, \quad \delta_a = (W_2^T \delta_z) \odot r.$$

Parameter	Gradient	Shape (column-vector convention)
W_2	$\delta_z h^T$	$C \times m$
b_2	δ_z	$C \times 1$
W_1	$\delta_a x^T$	$m \times d$
b_1	δ_a	$m \times 1$

For example, $dz = dW_2 h + W_2 dh + db_2$; taking the inner product with δ_z produces the outer product $\delta_z h^T$. The same argument applied to da gives $\delta_a x^T$.

Q1(b–c) · Numerical gradients and verification

Using the second class (PyTorch label 1), the supplied parameters give

```
a = [1, -1]^T     h = [1, 0]^T     z = [1, -1]^T
p = [0.880797077978, 0.119202922022]^T
loss = log(1 + exp(2)) = 2.126928011043
```

Let $q = \exp(2)/(\exp(2) + 1) = 0.880797077977882$. Then $\delta_z = (q, -q)^T$, $W_2{}^T\delta_z = (2q, -2q)^T$, $r = (1, 0)^T$, and $\delta_a = (2q, 0)^T$. Therefore all four gradients are

```
grad W2 = [[ q, 0],       grad b2 = [ q, -q]^T
           [-q, 0]]

grad W1 = [[2q, -4q],    grad b1 = [2q, 0]^T
           [ 0,   0]]
```

Gradient	Numerical entries
$\nabla_{W_2}\ell$	[[0.880797077978, 0], [−0.880797077978, 0]]
$\nabla_{b_2}\ell$	$[0.880797077978, -0.880797077978]^T$
$\nabla_{W_1}\ell$	[[1.761594155956, −3.523188311912], [0, 0]]
$\nabla_{b_1}\ell$	$[1.761594155956, 0]^T$

Independent float64 check

verify_gradients.py evaluates the forward pass and analytic chain-rule gradients with NumPy float64. It separately creates PyTorch float64 leaf parameters, calls cross_entropy on the logits, and obtains each gradient with backward(). Every entry of all four parameter gradients is compared.

Measured maximum absolute difference: 0.0 (also 0.0 separately for W_1, b_1, W_2, and b_2). Rank-one arrays store the column-vector entries in the code; the outer-product formulas are unchanged.

Software and hardware used throughout

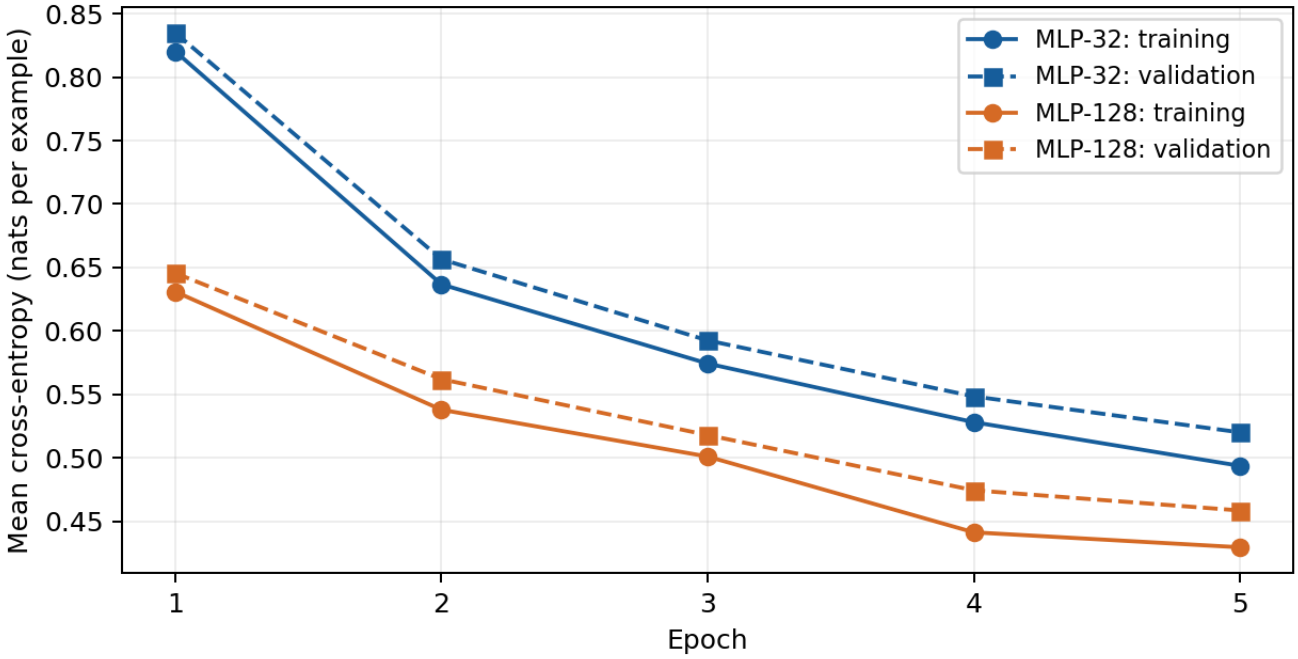
Python 3.12.14; PyTorch 2.6.0+cpu; torchvision 0.21.0+cpu; NumPy 2.3.5. Linux x86-64 CPU, Intel Xeon Platinum 8573C; five vCPUs available, two PyTorch threads per experiment. Q1 uses float64; training and image inference use float32. No GPU was used.

Q2(a) · Training the two MLPs

Architecture: Flatten → Linear(784, H) → ReLU → Linear(H, 10), independently for H = 32 and H = 128. The implementation explicitly clears gradients, computes raw-logit cross-entropy, backpropagates, and takes an Adam step.

The assignment's load_data function fixes 10,000 training and 2,000 validation images with NumPy seed 2026; the complete official 10,000-image test split is retained. Pixels are float32 in [0, 1], labels are int64. Each run uses five epochs, batch size 128, Adam(lr=0.001, weight_decay=0), default initialization and biases. Before constructing each model, torch.manual_seed(2026) is reset and a fresh shuffled DataLoader uses its own generator seeded 2026, num_workers=0 and drop_last=False.

At each epoch's end, non-shuffled loaders evaluate the full training and validation splits under eval() and no_grad(), with losses weighted by example count. The minimum validation cross-entropy selects a copied checkpoint (earliest epoch breaks ties). All three model runs and checkpoint selections finish before the test set is evaluated once per model. There is no augmentation, extra normalization, dropout, or batch normalization.



End-of-epoch losses, evaluated after training each epoch; both widths share the same axes.

Model	Parameters	Selected epoch	Train accuracy	Validation accuracy	Test accuracy
MLP-32	25,450	5	83.24%	83.20%	81.28%
MLP-128	101,770	5	85.11%	84.25%	82.82%
CNN	5,146	5	79.04%	79.25%	77.80%

The CNN row is included here for the combined Q2/Q3 comparison; its architecture and curves are discussed below. All selected checkpoints are from epoch 5. Complete per-epoch measurements and protocol event order are saved in results/fashion_results.json.

Q2(b) · Parameters and the effect of width

Including both layers' biases, the input-to-hidden layer contributes 784H + H parameters and the hidden-to-output layer contributes 10H + 10. Hence

$$P(H) = 784H + H + 10H + 10 = \mathbf{795H + 10}.$$

Width	Formula	PyTorch parameter sum
H = 32	795 × 32 + 10 = 25,450	25,450
H = 128	795 × 128 + 10 = 101,770	101,770

sum(p.numel() for p in model.parameters()) verifies both counts. Widening increases the parameter count by 3.999×. It lowers selected training cross-entropy from 0.4938 to 0.4296, and validation cross-entropy from 0.5202 to 0.4586. Training accuracy improves by 1.87 percentage points and validation accuracy by 1.05 points; test accuracy rises by 1.54 points.

The selected training-minus-validation accuracy gaps are 0.04 percentage points for H = 32 and 0.86 for H = 128. Both training and validation loss decrease throughout the five epochs. The wider model's slightly larger gap is small, and its validation loss does not turn upward, so this run does not show clear overfitting. The narrower model's higher training and validation losses are consistent with greater underfitting relative to the wider model, although five still-improving epochs cannot separate limited capacity from incomplete optimization or establish either model's asymptotic behavior.

One next experiment — proposed, not run

Train both widths for 20 epochs with all other settings unchanged, selecting only by validation cross-entropy and leaving the test set untouched. This would show whether the narrow model catches up with more optimization and whether the wider model eventually develops a widening training–validation loss gap.

Measured losses behind the MLP curves

Epoch	H=32 train CE	H=32 val CE	H=128 train CE	H=128 val CE
1	0.820207	0.834885	0.630950	0.645604
2	0.636749	0.656505	0.537981	0.562066
3	0.574372	0.592572	0.501112	0.518037
4	0.528164	0.548495	0.441420	0.474542
5	0.493806	0.520183	0.429639	0.458639

Q3(a) · CNN shapes and parameter counts

For a spatial input dimension n, kernel k, padding p, dilation d and stride s, convolution or pooling produces

$$n_{out} = \text{floor}((n + 2p - d(k - 1) - 1)/s + 1).$$

Convolutions use stride 1 and dilation 1. ReLU leaves the shape unchanged; both max pools use a 2 × 2 kernel and stride 2.

Stage	Spatial calculation	Output shape
Input	—	B × 1 × 28 × 28
Conv1 + ReLU	(28 − 5)/1 + 1 = 24	B × 6 × 24 × 24
Max pool 1	floor((24 − 2)/2 + 1) = 12	B × 6 × 12 × 12
Conv2 + ReLU	(12 + 2 × 1 − 3)/1 + 1 = 12	B × 12 × 12 × 12
Max pool 2	floor((12 − 2)/2 + 1) = 6	B × 12 × 6 × 6
Flatten	12 × 6 × 6 = 432	B × 432
Linear classifier	432 → 10 logits	B × 10

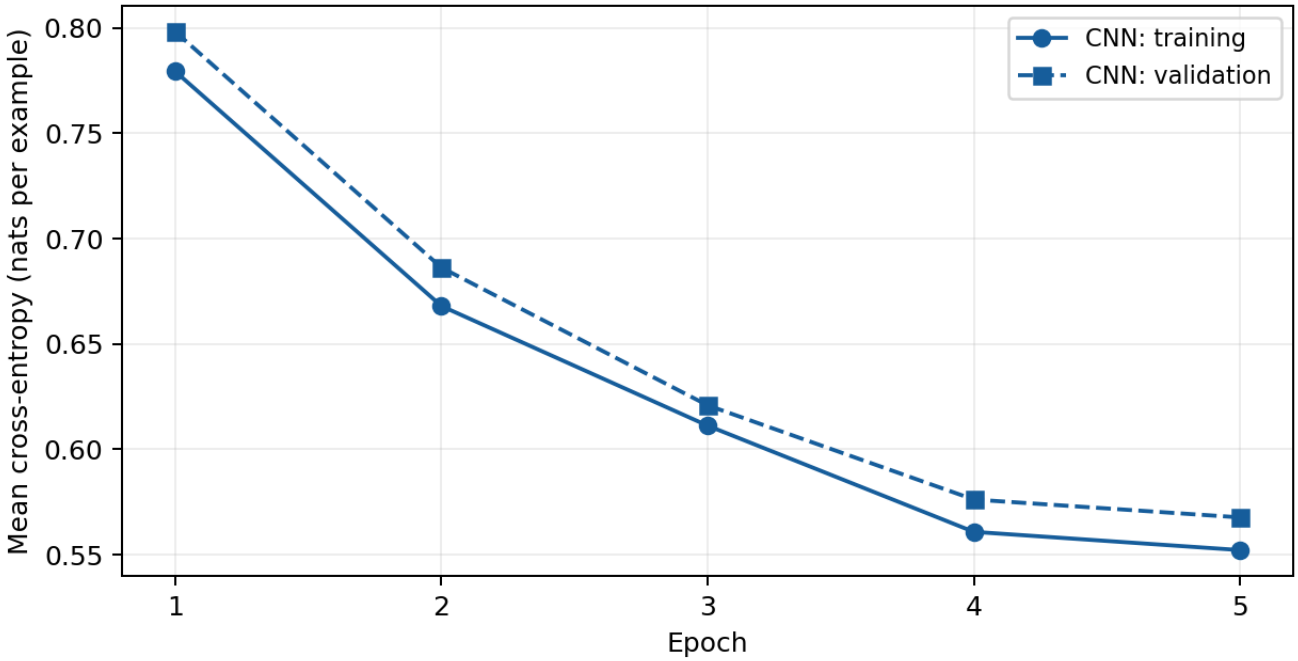
Learned layer	Weights plus biases	Parameters
Conv1	6(1 × 5 × 5 + 1)	156
Conv2	12(6 × 3 × 3 + 1)	660
Classifier	10(432 + 1)	4,330
Total	156 + 660 + 4,330	5,146

A dummy batch with B = 2 verifies every convolution, ReLU, pooling, flatten and classifier output shape. The total from sum(p.numel() for p in model.parameters()) is also 5,146, with the same layer-by-layer counts.

Each output-channel convolution filter and its bias are reused at every spatial position. Moving the filter produces distinct activations from local image patches, but does not create new learned parameters. Multiplying the filter parameter count by the output height and width would therefore overcount the shared weights.

Q3(b) · CNN training and comparison

The CNN was trained from scratch for five epochs with exactly the Q2 data splits, preprocessing, optimizer, seeds, loader settings, and checkpoint rule. Its selected checkpoint is epoch 5, with validation cross-entropy 0.567707.



Model	Parameters	Selected epoch	Train accuracy	Validation accuracy	Test accuracy
MLP-32	25,450	5	83.24%	83.20%	81.28%
MLP-128	101,770	5	85.11%	84.25%	82.82%
CNN	5,146	5	79.04%	79.25%	77.80%

Comparison with the validation-selected MLP

The comparison model is MLP-128 because its selected validation cross-entropy (0.458639) is lower than MLP-32's (0.520183), independently of their test scores. The CNN reaches 79.04% training, 79.25% validation and 77.80% test accuracy, compared with MLP-128's 85.11%, 84.25% and 82.82%, so it trails by 5.00 validation and 5.02 test percentage points in this run. It uses only 5,146 parameters versus 101,770 for MLP-128, about 19.78 times fewer. Local receptive fields and shared convolutional filters provide an efficient way to reuse edge and texture features across positions, whereas the MLP learns separate connections for each input pixel. Those useful inductive biases do not guarantee an advantage for this very small two-convolution architecture after only five epochs: both CNN losses are still falling and its tiny training–validation gap suggests incomplete fitting rather than clear overfitting.

The code records three checkpoint-selection events before the first test-evaluation event, and exactly one test evaluation for each selected checkpoint. No training setting was changed after seeing the test results.

Q4(a) · Three images selected before inference

The images, source URLs, SHA-256 hashes, and the descriptions below were saved in images/sources.json before running either classifier. The same three RGB images are used for both models; the bus scene is the harder case.

Dog (clear)



Cat (clear)



Bus (hard)



Dog · clear

A fluffy white dog with pointed ears sits on green grass, with dark foliage behind it. The dog is the clear main subject; its exact breed is not independently established.

Source: https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg

Cat · clear

A close-up of a brown-orange striped cat shows its face, green eyes, whiskers, and part of its chest. The image identifies a domestic cat clearly, but does not establish a breed.

Source: https://raw.githubusercontent.com/scikit-image/scikit-image/v0.19.3/skimage/data/chelsea.png

Bus · hard

A blue-and-white city bus is parked on a sunny street in front of a building. Several pedestrians partially occlude it, and bollards, pavement, signs, and windows add clutter; the bus is the main object of interest.

Source: https://raw.githubusercontent.com/ultralytics/assets/main/im/bus.jpg

Exact dog or cat breed is not independently known from these photographs. Correctness judgments below therefore distinguish a broad animal match from an unverified exact ImageNet label.

Q4(b) · ImageNet predictions

Both classifiers are torchvision models in eval() mode with IMAGENET1K_V1 weights; neither is trained or fine-tuned. Each RGB image is processed with its own weight object's transforms(), including resizing, center cropping and normalization. Probabilities are softmax(logits, dim=1), and labels are read from weights.meta["categories"].

ResNet-18: ResNet18_Weights.IMAGENET1K_V1. MobileNetV3-Small: MobileNet_V3_Small_Weights.IMAGENET1K_V1. Both input crops are 224 × 224; ImageNet normalization uses mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).

Image / model	Top three labels and softmax probabilities	Top-1 match and reason
Dog ResNet-18	1. Samoyed: 88.46% 2. Arctic fox: 4.58% 3. white wolf: 4.43%	**partly** — The dog category and visible appearance are plausible, but the exact Samoyed breed is not independently established.
Cat ResNet-18	1. Egyptian cat: 39.14% 2. tiger cat: 38.61% 3. tabby: 21.76%	**partly** — A domestic cat is present, but the predicted breed is not independently established.
Bus ResNet-18	1. minibus: 82.51% 2. police van: 11.52% 3. recreational vehicle: 0.83%	**partly** — It is a bus, but the image alone does not establish the more specific minibus subtype.
Dog MobileNetV3-Small	1. Samoyed: 75.79% 2. Arctic fox: 6.54% 3. Pomeranian: 6.52%	**partly** — The dog category and visible appearance are plausible, but the exact Samoyed breed is not independently established.
Cat MobileNetV3-Small	1. tiger cat: 49.31% 2. Egyptian cat: 29.66% 3. tabby: 20.92%	**yes** — The visible domestic cat has a striped tabby coat; this does not verify a breed.
Bus MobileNetV3-Small	1. minibus: 66.25% 2. police van: 20.16% 3. trolleybus: 5.91%	**partly** — It is a bus, but the image alone does not establish the more specific minibus subtype.

Comparison

Both models rank Samoyed first for the dog and minibus first for the hard street scene, with ResNet-18 assigning higher top probabilities (88.46% versus 75.79% for the dog; 82.51% versus 66.25% for the bus). They disagree on the cat: ResNet-18 ranks Egyptian cat first, narrowly above tiger cat, whereas MobileNetV3-Small ranks tiger cat first. The visible stripes support a domestic-cat/coat-pattern interpretation, but exact breeds are not independently established and the bus subtype remains uncertain. A higher softmax value does not guarantee correctness, and these three images do not establish a general model ranking.

Q4(c) · ResNet-18 Grad-CAM

Grad-CAM targets each image’s own ResNet-18 top predicted class logit, using target_layers = [model.layer4[-1]] and grad-cam 1.5.7. The CAM call runs with gradients enabled. Each overlay uses the exact model input crop, recovered by undoing its normalization.

ResNet-18 | dog | Samoyed (88.46%)

Exact model input crop



Grad-CAM overlay



The warmest region is around the dog's face, ears, and upper neck, with much cooler grass, foliage, and lower body. This aligns with salient dog features and helps interpret the animal prediction. It does not independently establish the Samoyed breed.

ResNet-18 | bus | minibus (82.51%)

Exact model input crop



Grad-CAM overlay



The strongest red region covers the bus's front/window area, with yellow support extending along its upper side windows; the pavement and lower pedestrians are relatively blue. The coarse map also overlaps part of a pedestrian, so it suggests vehicle-related cues without isolating the bus precisely. It helps explain the broad bus prediction but does not verify the specific minibus subtype.

Red indicates stronger and blue weaker support within an individual map. These coarse, separately normalized maps are visual clues, not segmentation masks or proof of causation; their colors do not compare confidence across images.

Run instructions and references

Reproducing the results

The supplied Python files implement the derivation check, both MLPs, CNN, custom training and evaluation loops, pretrained prediction, and Grad-CAM. README.md contains environment creation and the full commands. Run scripts from this solution directory, keeping the external data/weight cache separate from the submitted files.

```
python verify_gradients.py
python train_fashion.py --cache-dir /path/to/cache
python pretrained_cam.py --cache-dir /path/to/cache
python make_report.py
```

The result JSON files retain the measured per-epoch metrics, checkpoint-selection order, complete prediction probabilities, and numerical-gradient comparisons. Downloaded Fashion-MNIST data and model checkpoints are intentionally excluded from the solution package. A new run can differ slightly across platforms and PyTorch versions despite the fixed seeds.

Dataset download note

The default Fashion-MNIST HTTP mirror was unavailable in this environment. The loader retried the official dataset's HTTPS GitHub mirror at raw.githubusercontent.com/zalandoresearch/fashion-mnist/master/data/fashion/. It continued to use torchvision.datasets.FashionMNIST and its unchanged file MD5 checks; the dataset, split, and preprocessing were not changed.

Sources and adapted examples

[1] Supplied EE4745_HW2_Fall2026.pdf, pages 1–6. The page-6 load_data function and DataLoader settings are reproduced/adapted, and the page-5 Grad-CAM guide is followed. The models, training and evaluation loops, prediction code, and verification code were written for this solution.

[2] PyTorch, Datasets & DataLoaders.
https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html

[3] PyTorch, Optimizing Model Parameters.
https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[4] torchvision model and weight APIs.
https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html
https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.mobilenet_v3_small.html

[5] Jacob Gildenblat et al., pytorch-grad-cam, Usage examples and Choosing the layer(s). The GradCAM context-manager example and ClassifierOutputTarget/show_cam_on_image interface are adapted.
https://github.com/jacobgil/pytorch-grad-cam#usage-examples

[6] Image sources are given in Q4(a) and images/sources.json: pytorch/hub dog.jpg, scikit-image chelsea.png, and ultralytics/assets bus.jpg. SHA-256 values identify the exact downloaded files.

[7] Official Fashion-MNIST repository.
https://github.com/zalandoresearch/fashion-mnist